1)

```
module test;
   bit [11:0] my_array [4];

initial begin
  my_array = '{12'h012, 12'h345, 12'h678, 12'h9AB};

   for (int i=0;i<$size(my_array); i++) begin
      $display("Bits [5:4] of %b is %b", my_array[i], my_array[i][5:4]);
   end

   $display("--------------");

 foreach (my_array[i]) begin
      $display("Bits [5:4] of %b is %b", my_array[i], my_array[i][5:4]);
   end

end
endmodule // test
```

2) Same exclusions apply as in assignment 1

```
package alu_pkg;

// Create an enumerated type of the opcodes, opcode_t
typedef enum bit [1:0] {Add, Sub, Not_A, ReductionOR_B} opcode_t;

class alu_inputs;

  rand reg signed [3:0] A, B;
  rand opcode_t opcode;
  rand bit reset;

  constraint rst_con {
    reset dist {1:= 2, 0:= 98};
  }

endclass

endpackage
```

Kareem Waseem

```
import alu_pkg::*;

module ALU_4_bit_tb;

// inputs to instantiated ALU
reg clk, reset;
reg  signed [3:0] A; // Input data A: 2's complement
reg  signed [3:0] B; // Input data B: 2's complement

// Output of ALU
wire signed [4:0] C; // ALU output: 2's complement

integer       error_count; // 32-bit signed
integer       correct_count; // 32-bit signed

// Create a variable, opcodes, of type opcode_t
opcode_t opcode;

ALU_4_bit DUT (
      .clk(clk),
      .reset(reset),
      .Opcode(opcode),   // The opcode
      .A(A),    // Input data A: signed
      .B(B),    // Input data B: signed
      .C(C)        // ALU output: signed
      );

// Create the clock and reset

initial begin
   clk = 0;
   forever #1 clk = ~clk;
end

alu_inputs alu_obj = new();

// Create the Opcode and A and B
initial begin
   error_count = 0;
   correct_count = 0;

   assert_reset;
```

Kareem Waseem

```
   // Testing alu
   repeat (10000) begin
      assert(alu_obj.randomize());
      A = alu_obj.A;
      B = alu_obj.B;
      reset = alu_obj.reset;
      opcode = alu_obj.opcode;
      case (opcode)
         Add:            check_result(A+B);
         Sub:            check_result(A-B);
         Not_A:          check_result(~A);
         ReductionOR_B:  check_result(|B);
      endcase
   end

   $display("%t: At end of test error count is %0d and correct count = %0d", $time, error_count, correct_count);
   $finish;
end

task check_result (input signed [4:0] expected_result);
   @(negedge clk);
   if (reset)
      checkC();
   else begin
      if (expected_result !== C) begin
         error_count = error_count + 1;
         $display("%t: Error: For Opcode = %0b, A= 0d%0d, and B=0d%0d C should equal 0d%0d but is 0d%0d ", $time, Opcode, A, B, expected_result, C);
      end
      else
         correct_count = correct_count + 1;
   end
endtask

task checkC();
   @(negedge clk);
   if (C !== 0) begin
     error_count = error_count + 1;
     $display("%t: Error: C should equal 0d0 but the calcuated result is 0d%0d ", $time, C);
   end
endtask // checkC

task assert_reset;
   reset = 0;
   @(negedge clk);
   reset = 1;
   repeat(5) @(negedge clk);
   // Check reset value of C
   checkC();
   reset = 0;
endtask // assert_reset
endmodule
```

3) No exclusions needed.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FSM_1	When the reset is asserted, the output should be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.	-	A checker in the testbench to make sure the output is correct
FSM_2	When the pattern 010, is detected, y output should be high and users_count increment	Randomization under constraints on x to be 0 67% of the simulation time	-	Check the output against golden model

```
package FSM_pkg;
typedef enum {IDLE, ZERO, ONE, STORE} state_e;

class fsm_transaction;
rand bit x, rst;
bit y_exp;
bit [9:0] users_count_exp;

// FSM_1
constraint rst_con {
 rst dist {1:= 2, 0:= 98};
}

// FSM_2
constraint x_con {
 x dist {0:= 67, 1:= 33};
}

endclass
endpackage
```

```
import FSM_pkg::*;
module FSM_010_tb;

logic x, y;
logic [9:0] users_count;
logic clk, rst;
fsm_transaction f_txn = new;

int error_count;
int correct_count;

FSM_010 f1(clk, rst, x, y, users_count);

// Create the clock
initial begin
   clk = 0;
   forever #1 clk = !clk;
end

// Stimulus Generation
initial begin
  error_count = 0;
  correct_count = 0;
  x = 0;

  assert_reset;

  repeat(20000) begin
    assert(f_txn.randomize());
    x = f_txn.x;
    rst = f_txn.rst;
    check_result(f_txn);
  end

  // disable reset for user_count coverage
   repeat(10000) begin
     assert(f_txn.randomize());
     x = f_txn.x;
     rst = 0;
     f_txn.rst = 0;
     check_result(f_txn);
  end
```

Karim Waseem

```
  $display("%t: At end of test error count is %0d and correct count = %0d", $time, err
  $stop;
end

// Checker
task check_result(input fsm_transaction f_txn_c);
   golden_model(f_txn_c);
   if (f_txn_c.rst !== 1) begin
      @(negedge clk);
      if ( (f_txn_c.y_exp !== y) || (f_txn_c.users_count_exp !== users_count) ) begin
        error_count = error_count + 1;
        $display("%t: Error: For x= 0d%0d, rst=0d%0d, y should equal 0d%0d but is 0d%0
      end
      else
        correct_count = correct_count + 1;
   end
   else begin //rst check
      check_rst;
   end

endtask

// Golden Model
task golden_model(input fsm_transaction f_txn_g);
   state_e cs, ns;
   case (cs)
      IDLE: begin
         if (f_txn_g.x)
            ns = IDLE;
         else
            ns = ZERO;
      end
```

```
   ZERO: begin
      if (f_txn_g.x)
         ns = ONE;
      else
         ns = ZERO;
   end

   ONE: begin
      if (f_txn_g.x)
         ns = IDLE;
      else
         ns = STORE;
   end

   STORE: begin
      if (f_txn_g.x)
         ns = IDLE;
      else
         ns = ZERO;
   end
endcase
@(posedge clk);
if (f_txn_g.rst) begin
   cs <= IDLE;
   f_txn_g.users_count_exp <= 0;
end
else begin
   //sequential output
   f_txn_g.users_count_exp <= (cs == STORE)? f_txn_g.users_count_exp+1 : f_txn_g.users_count_exp;
   cs = ns;
end
```

```
   //Comb output
   f_txn_g.y_exp = (cs == STORE)? 1:0;
endtask

task assert_reset;
   //assert reset
   rst = 1;
   check_rst;
   //desert reset
   rst = 0;
endtask

task check_rst;
   @(negedge clk);
   if (users_count !== 0 && y !== 0) begin
     error_count = error_count + 1;
     $display("%t: Error: Reset value is asserted and the output is not tied low", $time);
   end
   else
     correct_count = correct_count + 1;
endtask
endmodule
```